

Microservices Lab – Implementation Documentation

1. Project Overview

The objective of this lab was to deploy a microservices-based application using AWS services and Docker.

The application was divided into two microservices:

- Customer Microservice
- Employee Microservice

The microservices were containerized using Docker, stored in Amazon ECR, and deployed using Amazon ECS. An Application Load Balancer was used to distribute traffic to the services.

AWS CodeCommit, CodePipeline, and CodeDeploy were used to implement the CI/CD workflow.

2. Implementation Steps

Step 1: Open AWS Cloud9

AWS Cloud9 was used as the development environment for the project.

The Cloud9 environment provided a terminal for running Git, Docker, and AWS CLI commands.

The environment was opened and used to access the project source code and perform the required deployment operations.

Step 2: Access the Microservices Repository

The application source code was maintained in AWS CodeCommit.

The repository named microservices was used for the application source code.

The repository contained the application components required for the Customer and Employee microservices.

The source code was accessed from the Cloud9 environment for further processing.

Step 3: Separate the Application into Microservices

The application was organized into two independent services:

Customer Microservice

The Customer Microservice handles customer-related functionality.

Employee Microservice

The Employee Microservice handles employee and supplier-related functionality.

Separating the application into two services allows each service to be managed and deployed independently.

Step 4: Create Amazon ECR Repositories

Amazon Elastic Container Registry (ECR) was used to store the Docker images.

Two repositories were created:

customer
employee

The customer repository stores the Customer Microservice image, while the employee repository stores the Employee Microservice image.

Step 5: Build the Docker Images

Docker was used to containerize the microservices.

The Docker image for each microservice was created using its respective Dockerfile.

For example:

`docker build -t customer` and `docker build -t employee` .

The created images were verified using: `docker images`

This confirmed that the required container images were available locally.

Step 6: Authenticate Docker with Amazon ECR

Docker was authenticated with the Amazon ECR registry using the AWS CLI.

The authentication command follows the format:

```
aws ecr get-login-password --region <region> | docker login --username AWS  
--password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com
```

After successful authentication, the Docker images could be pushed to the ECR repositories.

Step 7: Tag and Push the Docker Images

The Docker images were tagged with their respective ECR repository addresses.

The Employee image, for example, was tagged using the Employee ECR repository.

```
docker tag employee:latest <ECR-Repository-URI>:latest
```

The image was then pushed to Amazon ECR:

```
docker push <ECR-Repository-URI>:latest
```

The same process was followed for the Customer Microservice.

The successful push confirmed that the container images were available in Amazon ECR.

Step 8: Create the Amazon ECS Cluster

Amazon Elastic Container Service (ECS) was used to run the containerized microservices.

An ECS cluster named:

```
microservices-serverscluster
```

was used for the application.

The cluster provides the environment in which the Customer and Employee services run as ECS tasks.

Step 9: Configure ECS Services

ECS services were configured for the microservices.

The services use the Docker images stored in Amazon ECR.

The ECS services manage the running tasks and ensure that the required containers are available.

The ECS console was used to verify the services and running tasks.

Step 10: Create Target Groups

Target groups were created to route traffic from the Application Load Balancer to the ECS tasks.

The configured target groups included:

customer-tg-one
customer-tg-two
employee-tg-one
employee-tg-two

The target groups were configured for HTTP traffic on port 80.

The target groups provide the connection between the load balancer and the ECS services.

Step 11: Configure the Application Load Balancer

An Application Load Balancer was created to provide access to the deployed application.

The load balancer was named: microservicesLB

The load balancer receives incoming requests and forwards them to the appropriate target group.

This allows users to access the application through a common endpoint instead of directly accessing individual ECS tasks.

Step 12: Configure Security Groups

Security groups were configured to control network access to the application resources.

The security groups were associated with the required AWS resources, including the load balancer and application components.

The required HTTP traffic was allowed so that requests could reach the deployed application.

Step 13: Create and Verify CloudFormation Resources

AWS CloudFormation was used to manage the infrastructure resources required for the application.

The CloudFormation console was used to verify the created stacks.

The stacks showed the status:

CREATE_COMPLETE

This indicates that the required CloudFormation resources were successfully created.

Step 14: Configure AWS CodeCommit

AWS CodeCommit was used for source-code management.

The CodeCommit console contained repositories including:
deployment
microservices

The microservices repository was used to store the application source code.

Changes to the application could therefore be managed through version control.

Step 15: Configure AWS CodePipeline

AWS CodePipeline was used to automate the application deployment process.

Separate pipelines were configured for the microservices:

update-employee-microservice
update-customer-microservice

The pipeline connects the source repository with the deployment process. The pipeline automatically processes changes and starts the required deployment stages. The pipeline executions were checked to verify that the deployment completed successfully.

Step 16: Deploy Using AWS CodeDeploy

AWS CodeDeploy was used to deploy the updated ECS services.

The deployment process created a replacement task set for the updated version of the application.

The deployment included the following stages:

1. Create the replacement task set.
2. Test the replacement task set.
3. Shift traffic to the replacement task set.
4. Complete the deployment.
5. Terminate the original task set.

After successful deployment, traffic was shifted to the replacement task set. The final deployment showed the replacement task set receiving the application traffic.

Step 17: Verify the Deployed Application

After completing the deployment, the application was accessed through the Application Load Balancer.

The deployed application displayed the: Coffee Suppliers interface.

The application provided access to supplier-related functionality and administrator features. This confirmed that the application was successfully deployed and accessible.

Step 18: Verify Supplier Management

The Employee Microservice provides supplier-management functionality.

The administrator page displayed the: Manage coffee suppliers interface.

The page provided options to:

- View supplier information
- Edit supplier information
- Add a new supplier

The supplier information was displayed successfully, confirming that the Employee Microservice was functioning correctly.

Step 19: Verify ECS Tasks

The ECS console was checked after deployment to verify that the microservices were running.

The ECS task list showed running tasks associated with:

customer-microservice

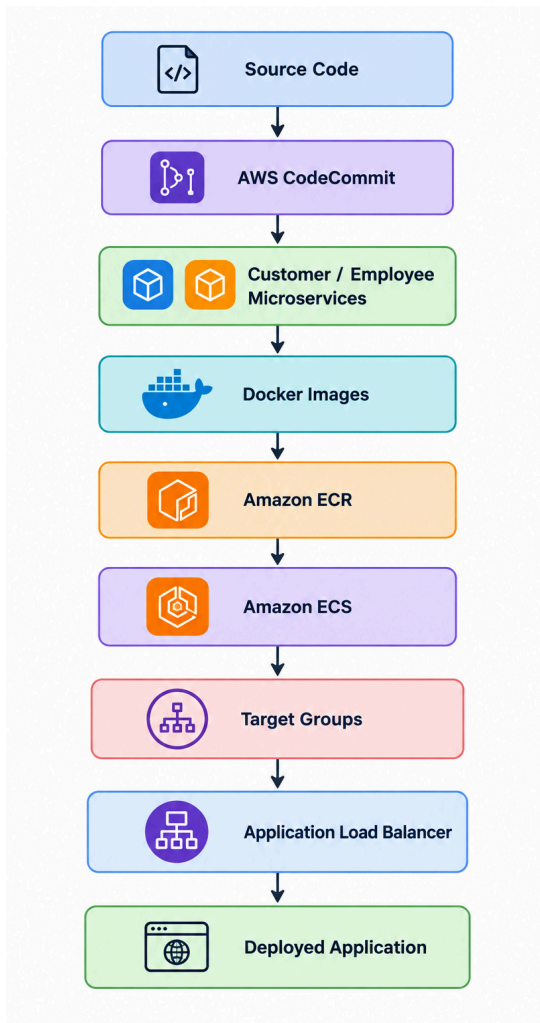
employee-microservice

The tasks were displayed in the Running state.

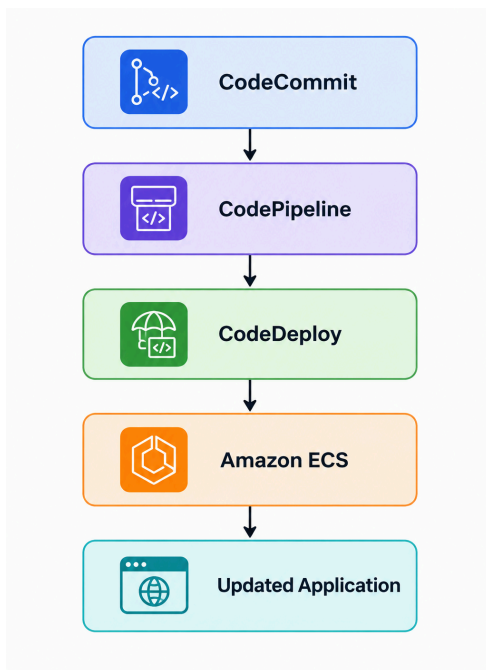
This confirmed that the containerized services were successfully deployed and running in the ECS cluster.

3. Final Workflow

The complete workflow followed in the lab was:



The CI/CD workflow was:



4. Result

The Customer and Employee microservices were successfully containerized and deployed using AWS.

The Docker images were stored in Amazon ECR and deployed through Amazon ECS. Target groups and an Application Load Balancer were configured to route application traffic.

AWS CodeCommit, CodePipeline, and CodeDeploy were used to implement the CI/CD workflow.

The deployed Coffee Suppliers application and supplier-management functionality were successfully verified, and the ECS console confirmed that the microservice tasks were running.

5. Conclusion

The lab provided practical experience in deploying a microservices application using AWS and Docker. It demonstrated containerization, container image management, ECS deployment, load balancing, source-code management, and CI/CD automation.

The successful deployment of the Customer and Employee microservices demonstrated the complete workflow from source code to a running application on AWS.